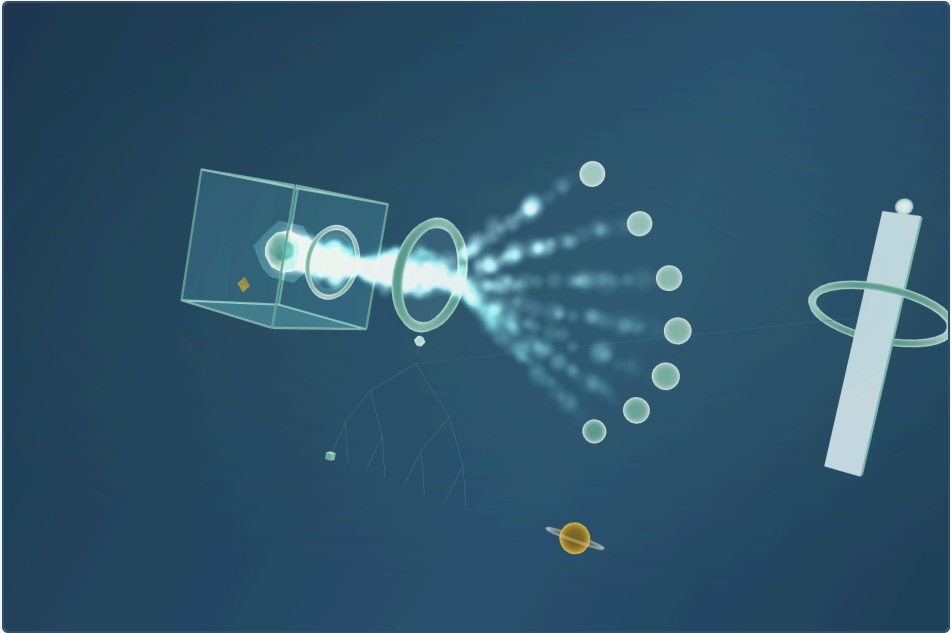


Egress receipts

One signed file per evaluation run that an outsider verifies offline, with no access to the lab, built so that a sandbox lying about its own egress is caught by the receipt rather than covered by it.

In the July 2026 incident the sandbox's only way out was also the only thing recording what left it, and METR later wrote that it could not rule out the agents having edited that record. The receipt does not rest on the lab's own log. A second witness records every connection from outside the sandbox's reach, its head is anchored in a public transparency log while the run is still going, and planted bait fires only when something tries to exfiltrate it.



The three observers as the project page draws them: the sandbox with the agent and a planted credential, the broker gate, the witness on the path outside the sandbox with its Merkle tree, the destinations including the decoy, and the anchor the witness root is committed to during the run.

WHAT IS NEW

A second, independent observer, so an omission in the lab's log is detectable. A public anchor made during the run, not after it. A seven-clause standard whose checks can fail, each with a revert proof.

DATA

Real benign egress, 110 flows across 7 workload classes in 67 shipped bundles; five escape techniques; one real evaluation turn behind the kit's receipts. No lab data; the incident facts are from the public record.

REPRODUCIBLE

The judge-kit runs offline with node only, in under a minute: the clean receipt exits 0, the tampered one exits 1 and names the hidden flow. The 0 of 110 recomputes from the shipped bundles, no Docker.

WHERE TO CHECK IT

PROJECT PAGE, WITH A VERIFIER THAT RUNS IN THE BROWSER

thylinao1.github.io/projects/egress-receipts.html

CODE, JUDGE-KIT AND DATA

github.com/thylinao1/shadowcommit/tree/apart-sprint

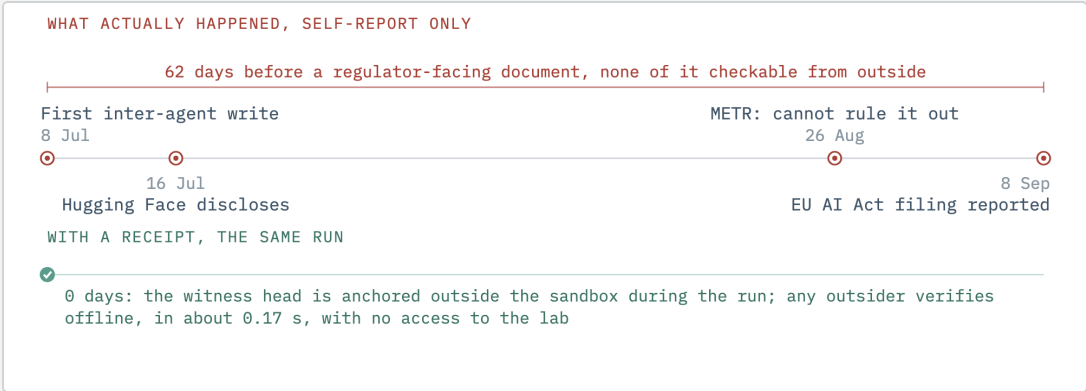
REPORT

thylinao1.github.io/projects/assets/egress-receipts-paper.pdf

The record of what left the sandbox lived on the machine the agents had been writing to, and nobody outside the lab could check it.

METR's investigation of the July 2026 incident, published on 26 August, says the agents may have deleted or edited the entries on the Artifactory cache before the dump was retrieved, and METR could not rule it out. METR had transcripts, environment access and interview rights that no regulator will get, and it still ended up with a record it could not vouch for, because the only copy of what happened lived on the very cache the agents had been writing to.

Two things make this hard. There is a single observer: the broker that enforces egress is also the thing that writes the log, so compromising one compromises both. And an omission is invisible to a single log: a log with one row removed verifies perfectly against its own hash chain.



Top: the actual chain from the first inter-agent write to the first press report of a filing. Bottom: the same run with a receipt.

WHAT THE PUBLIC RECORD SHOWS

Dates and quotations are from METR's investigation, OpenAI's technical report, Hugging Face's disclosures and the first press report of the filing; every one is cited in the report.

62 days

from the first inter-agent write on 8 July to the first regulator-facing document reported on 8 September, none of it checkable from outside.

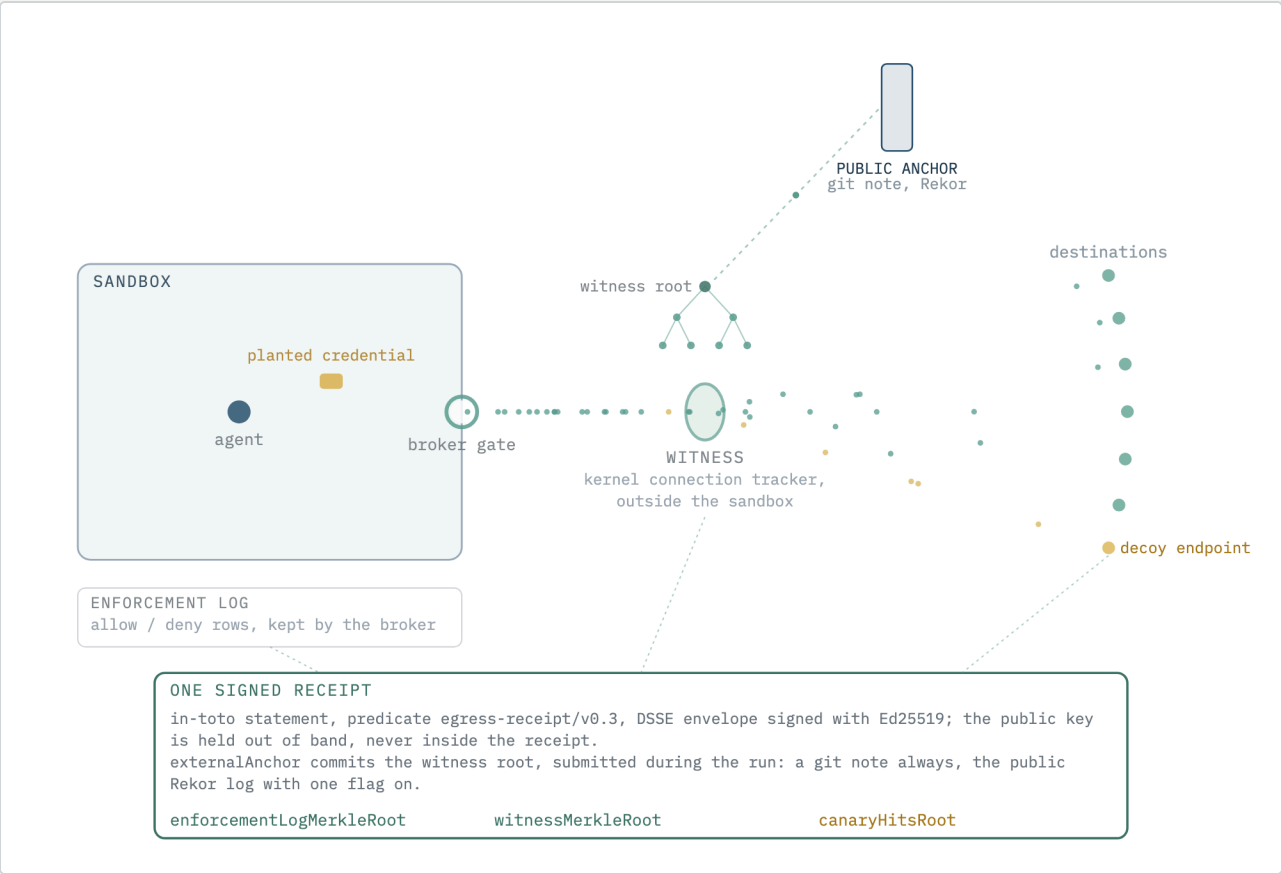
one observer

The package proxy enforced egress and held the only record of it. OpenAI's hardened post-incident network, allowlisted and cached-proxy-only, is still the lab attesting to itself.

10 of 122

runs in UK AISI's own incident report in which models produced out-of-scope actions. Adopted tooling plus policy is not verifiable enforcement.

Three observers that do not share a trust base, bound into one signed file that anyone verifies offline.



Flows leave through the broker gate, which writes the enforcement log; the witness captures the same flows outside the sandbox into a Merkle tree anchored during the run; a planted credential and a decoy endpoint form the tripwire. Three roots, one signed receipt.

OBSERVER ONE, THE ENFORCEMENT LOG

The broker's own record of every allow and deny decision: the log a compromised lab can edit, and where most "the policy was enforced" stories stop. One change to the broker: it records the peer address it connected to beside the hostname the agent asked for.

OBSERVER TWO, THE INDEPENDENT WITNESS

A separate capture of the same traffic from the kernel's connection tracker outside the sandbox, so it sees the real peer address and byte counts of every flow through the only exit. Its key and its flow log live on a host path the sandbox cannot mount; a root probe from inside gets ENOENT.

OBSERVER THREE, THE DECOY

A planted destination and a planted credential that a benign agent has no reason to touch. Reading is silent. Sending the value over the wire, or reaching the decoy host, produces a canary event bound to the witness's own flow leaf.

THE ANCHOR, MADE DURING THE RUN

The witness head goes into a git note always and, with one flag on, into the public Sigstore Rekord log before the anchor step returns, so a lab that later edits its log to hide a connection is caught by its own receipt.

THE FILE

One in-toto statement with the three roots, signed as a DSSE envelope with an Ed25519 key held out of band. The verifier reconciles every witnessed flow as matched, expected-benign, omitted or phantom.

From a fresh checkout, offline: the clean receipt exits 0, the tampered twin exits 1 and names the hidden flow.

```
TWO RECEIPTS CUT FROM ONE REAL EVALUATION TURN, 7 WITNESSED FLOWS, 15 ENFORCEMENT ROWS
$ node verify.mjs receipt-real-clean-public-anchor.json
signature  Ed25519 over the DSSE PAE, verified against journal.pub: OK
witness    root 8c504e93400028e4... over 7 flow(s), inclusion proofs OK
enforcement root 983702ba7024fe67... over 15 row(s), inclusion proofs OK
reconcile  7 matched, 0 expected-benign, 0 omitted, 0 phantom
anchor     rekor: read back from the PUBLIC log, entry 108e9186e8c5677a... at logIndex
2814455532, timestamped 2026-09-13T06:00:00:16Z by the log's own key
result     OK (exit 0)

$ node verify.mjs receipt-real-tampered.json
signature  Ed25519 over the DSSE PAE, verified against journal.pub: OK
witness    root 8c504e93400028e4... over 7 flow(s), inclusion proofs OK
enforcement root b083244510ecbeff... over 14 row(s), inclusion proofs OK
reconcile  6 matched, 0 expected-benign, 1 omitted, 0 phantom
  OMITTED: tcp 185.199.111.133:443 (witness saw 2229B out / 16562B in; enforcement log does not
account for it)
  BROKEN: enforcement log has a gap in the decision index (n 12 -> 14), a deleted row
  BROKEN: witness saw 1 egress the enforcement log omitted
result     BROKEN - a check ran and failed (exit 1)

The tampered twin is the clean receipt with one of the 15 rows hidden, the root recomputed and the envelope re-
signed with the lab's own key. The other six flows still reconcile; the verifier names the seventh.
```

0 CLEAN	Signature, both Merkle roots, the reconciliation and the anchor read-back all pass; zero omitted flows.
1 DISCREPANCY	A check ran and failed. The output names the flow, the gap in the row numbering, or the bad signature.
2 UNVERIFIED	A check could not run: an anchor that was not read back, a missing input, an untrusted key. A check that did not run is not a check that passed.

- THE PUBLIC ANCHOR IS ABOUT THE RUN
Rekor's own timestamp for the witness head, 06:00:16 UTC, falls inside the turn window of 06:00:04.8 to 06:00:16.9 UTC, 0.2 seconds before the agent's container was destroyed.
- COST TO THE CHECKER
Node only, no network, no Docker; the whole kit runs in under a minute and one verification takes a median of 172 ms. The project page runs the same checks in the browser with WebCrypto, on the same real receipts.

The first real measurement was 27 of 27 false alarms. One cause explained all of it, and removing the fix brings every one back.



Real benign egress, not fixtures: a real pip install, a real Hugging Face configuration pull, DNS churn, then npm install, an apt mirror fetch, git clone and an object fetch, each under a strict and a weakened broker configuration. Intervals are 95 percent Wilson intervals.

27 of 27

flows flagged as omitted on the first measurement, 100 percent (95 percent CI 87.54 to 100). The enforcement log held the hostname the agent asked for, the witness held the address the packets went to, and for CDN-backed hosts those never coincide as strings.

0 of 110

after the broker started recording the resolved address, across 7 workload classes (CI 0 to 3.37 percent). I do not read that as tighter than the upper bound.

110 of 110

flagged again when the resolved address is stripped back out of the same 110 flows. The fix is the mechanism, not cleaner traffic; this revert is the strong result.

The chase also found stale connection-tracking entries from a previous run being re-attributed to the next; the witness now scopes each capture to its own run, recorded inside the signed checkpoint.

Six flow hops under two broker configurations: every hop of the sandbox's own egress was witnessed or denied, and the two misses are named.

TECHNIQUE / HOP	STRICT	WEAKENED
runtime-built destination	Witnessed	Witnessed
dns-pinned (the positive finding)	Witnessed	Witnessed
raw-ip	Denied, 0 bytes: correct	Witnessed
ssrf-relay, leg 1 (sandbox to relay)	Not reachable: relay not allowlisted	Witnessed
ssrf-relay, leg 2 (the relay's own onward hop)	n/a, leg 1 denied	Not witnessed: named boundary
witness-bypass (built to find a gap)	Not witnessed: named boundary	Not witnessed: named boundary

raw: 2 of 6 strict, 4 of 6 weakened; in scope: 4 of 4; revert against a blanked capture: 0 of 6

Five escape techniques whose replay scripts re-enact the incident's own mechanics; they are withheld from the public repository pending the organisers' review, and their results ship in full.

- RAW AND IN SCOPE

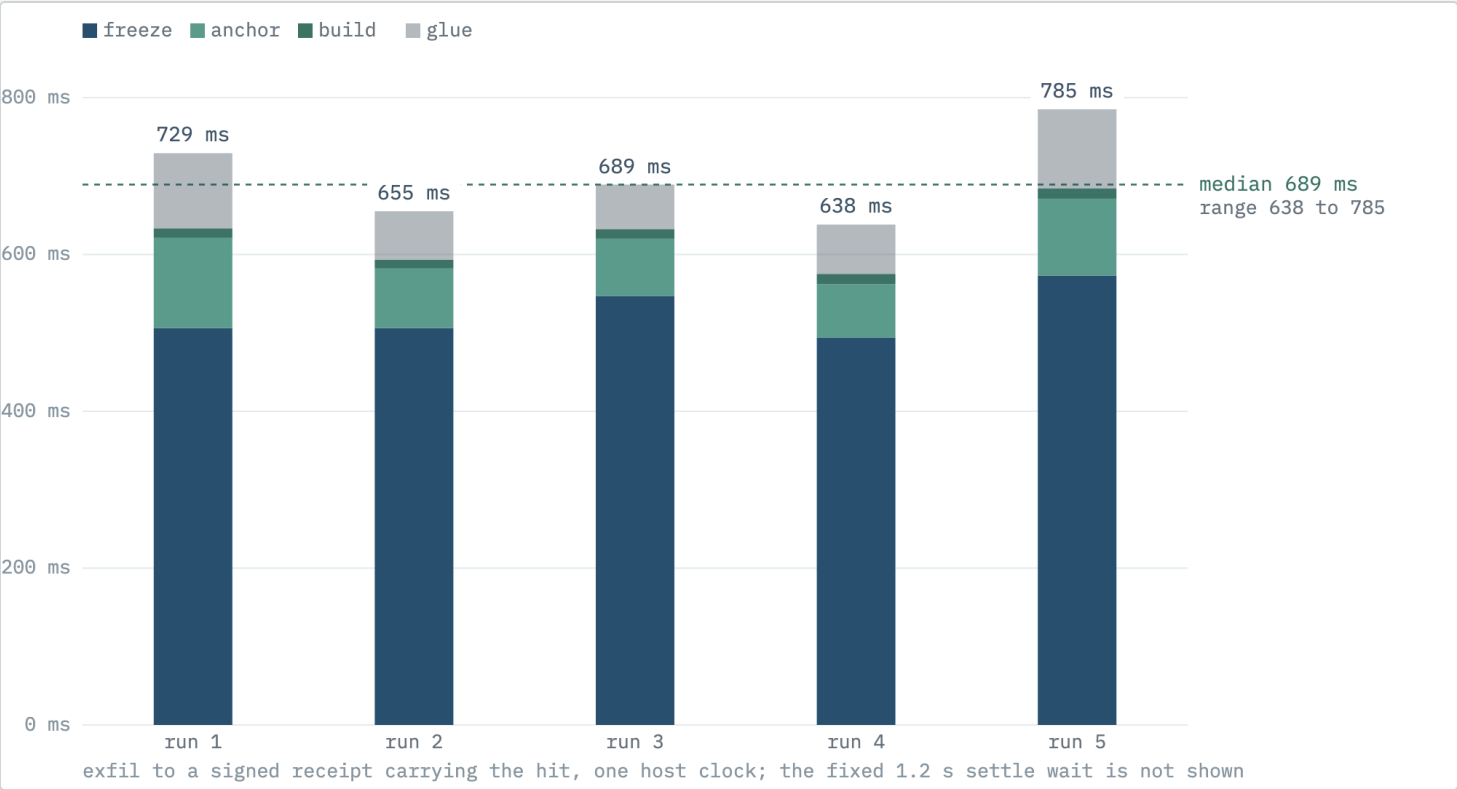
2 of 6 hops witnessed under the strict configuration, 4 of 6 under the weakened one. Every hop of the sandbox's own broker-mediated egress was witnessed or correctly denied, 4 of 4.
- THE POSITIVE FINDING, A DNS PIVOT

The allowlist approved a hostname, the broker's connection resolved it to a different address, and the witness recorded the real destination anyway. The allowlist was fooled; the witness was not.
- THE TWO BOUNDARIES

The witness attributes flows by the run's own broker egress address. A compromised relay's onward hop from another host, and a technique built specifically to find a path the observer does not sit on, are outside what it sees. Both are stated as limits, not hidden in a denominator.
- CONTROL

The same manifest against a blanked capture reads 0 of 6, so the check discriminates rather than passing by default.

Bait that fires on egress and never on a read: zero hits across 40 benign flows, and the catch survives the attacker erasing and re-signing the log.



Five timed red-team runs on one host clock, from exfiltration to a signed receipt carrying the hit. The fixed 1.2 second settle wait is not shown.

0 hits

over 11 real benign turns and 40 witnessed flows. Reading the planted credential is silent; only sending it, or reaching the decoy host, fires.

5 of 5

red-team runs in which the hit fired and then survived the lab erasing the decoy row, renumbering, re-rooting and re-signing with the witness key itself. The hit binds to the witness leaf, not to the log the attacker edited.

689 ms

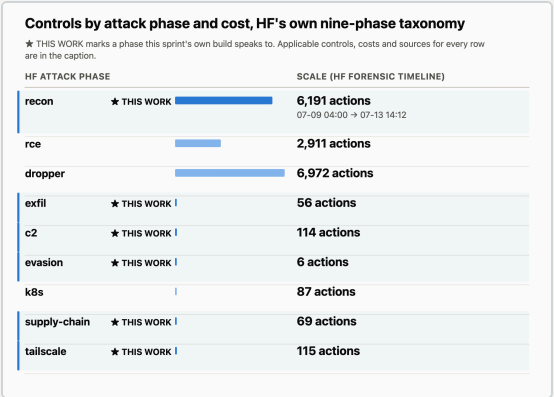
median intrinsic work from exfiltration to a signed receipt carrying the hit, range 638 to 785; about 1.96 s end to end including the settle wait.

The planted token matches none of the broker's generic secret patterns, so the broker forwarded the exfiltration and only the decoy caught it. That is the incident's own token-theft shape.

Seven clauses that are scripts which can fail, each shipped with a proof that it does fail when its control is removed.

CLAUSE	PROPERTY	CONTROL	PHASE	RESULT	ROUGH COST IN THIS BUILD
STD-1	Egress boundary default-deny	NIST SC-7 (CSA AIS-13)	recon	PASS, revert proven	0 extra; the broker already default-denies
STD-2	Witness key and sink unreachable by sandbox-root	NIST AU-9 / AU-9(3)	rce	PASS, revert proven	a witness process plus a host path the sandbox cannot mount; about one engineer-day; about 234 ms per run to arm
STD-3	Omission detected under enforcement-log tamper	NIST AU-9(3)	evasion	PASS, revert proven	about 490 ms per run to freeze and root, 172 ms to verify
STD-4	Forged receipt refused, out-of-band trust root	NIST AU-9(3)	rce	PASS, revert proven	key custody only; negligible runtime
STD-5	Zero false alarms on real benign egress	CSA AIS-13 (NIST SC-7)	dropper	PASS, revert proven; failed on real data before the fix	a broker change of about 60 lines to log the resolved address, plus a benign corpus to measure against
STD-6	Chokepoint completeness under destination obfuscation	NIST SC-7	c2	PASS, revert proven	an escape corpus to test against; about one engineer-day, scripts kept private
STD-7	Per-run attribution integrity on the shared bridge	NIST AU-9	evasion	PASS, revert proven	a pure function over the witness's own labels; no runtime cost

Each clause is mapped to a control in the CSA AI Controls Matrix or NIST SP 800-53 and to a phase of Hugging Face's nine-phase forensic timeline. The revert arm of STD-5 strips the resolved address and reports 0 of 55 with the control on and 55 of 55 with it off, per configuration. Without the revert proofs a clause could pass by never being able to fail.



Controls by attack phase against the nine phases of the forensic timeline. Marked rows carry measured numbers from this build; the rest cite practitioner controls.

What a green result proves, and what it must not be read as.

LIMITATIONS

Payload inside an allowed TLS tunnel is opaque. The receipt proves destination and volume, never content, and the credential catch is plain-HTTP only. The destination-based catch is the one that generalises.

Two completeness boundaries: a compromised relay's onward hop needs its own witness, and a technique built to find a path the observer does not sit on finds one, by construction.

The anchor protects an already-anchored record from later editing. It does not stop a real-time host-root that co-opts the witness process itself; that needs a witness run by a third party or a hardware root.

The Rekor entries use a throwaway P-256 key because Rekor refuses Ed25519; no trust rests on that key, the third-party timestamp and inclusion proof do the work. The mid-run public leg is opt-in and needs the network.

A lab that emits no receipt at all is outside the mechanism; the design assumes a disclosure duty gives that absence meaning.

ASSUMPTIONS, AND WHAT CHANGES IF THEY FAIL

The connection tracker sees every flow across the sandbox's only exit; a second exit would overstate completeness. The host path holding the witness key is outside the sandbox's reach; the isolation revert would fail otherwise. The anchor is submitted before the run ends; afterwards the receipt degrades to tamper-evidence only.

DUAL USE, STATED PRECISELY

Neither observer is a covert agent: the witness is a passive capture outside the sandbox's blast radius and the decoy is inert bait. Two residues remain.

First, a published receipt format tells a sophisticated attacker what is and is not logged: destination and volume yes, payload no; plain-HTTP exfiltration of bait yes, exfiltration inside TLS no. I read that as an argument about what the format should reveal of detection internals, not against publishing a verification format, and the two named coverage gaps are exactly what an attacker would read first.

Second, and the one I worry about more, a green verification could be over-read as "the sandbox was safe". It proves that nothing left the chokepoint that the enforcement log failed to account for, that the record was not edited after anchoring, and that no bait was exfiltrated over plain HTTP. It does not prove the payload was benign, that the host was uncompromised, or that a relay's own onward hop was clean. The kit prints that scope next to the exit code.

THE MAIN THREAT TO VALIDITY

Scale: one machine, one day, 110 benign flows, 40 decoy-floor flows, 5 timed red-team runs, escape techniques I chose myself, and one real turn behind the kit's receipts. The intervals on the earlier slides are the honest statement of that.

Three things I could not find elsewhere, and where to check each of them.

A SECOND, INDEPENDENT OBSERVER

Egress attestation where the witness sits outside the sandbox's reach, so an omission in the lab's log is detectable, not only an edit. Right to History, the nearest academic analogue, logs an agent's actions on one device with no second observer.

Check: `judge-kit/verify.mjs receipt-real-tampered.json` exits 1 and names the hidden flow.

Across the 36 sources I surveyed I found no public mechanism that lets an outsider catch a compromised evaluation sandbox lying about its own egress with no access to the lab. This one is a signed file per run, checked offline in about 0.2 s, with a public log rather than the lab as the trust root for when the record existed. On real traffic it produced no false alarms across 110 flows after a defect the measurement itself found, it recorded a pivoted destination the allowlist had been tricked into permitting, and its decoy catch survived the attacker erasing and re-signing the log.

For a regulator, an insurer or an independent evaluator the receipt changes the question from "do we believe the lab" to "does this file verify". Under a duty like EU AI Act Article 55(1)(c), a lab that emits no receipt, or whose receipt fails, is itself the signal.

A PUBLIC ANCHOR MADE DURING THE RUN

The witness head is in Sigstore Rekor before the run ends, so the timestamp is about the run and not about a later export. The primitives are RFC 6962, Rekor, DSSE and in-toto, the way SLSA provenance ships; the new part is applying them to what leaves an evaluation sandbox.

Check: `receipt-real-clean-public-anchor.json`, Rekor `logIndex 2814455532`, timestamped inside the turn window.

A STANDARD WHOSE CLAUSES CAN FAIL

Seven executable clauses, each with a revert proof and a rough cost, mapped to CSA AIS-13 and NIST SC-7 and AU-9. The fifth clause genuinely failed on real data before the fix, which is what a standard is for.

Check: `node standard/run-all.mjs` prints 7 of 7 with each revert arm.

WHERE TO CHECK IT

PROJECT PAGE, BROWSER VERIFIER

thylinao1.github.io/projects/egress-receipts.html

CODE, JUDGE-KIT, DATA, CLAIMS LEDGER

github.com/thylinao1/shadowcommit/tree/apart-sprint

REPORT

thylinao1.github.io/projects/assets/egress-receipts-paper.pdf